

Noctem 0.9.4 Specifications

1. CURRENT CAPABILITIES AND MECHANISMS (CODE-CONFIRMED)

2.1 Web/API control plane

- noctem/web/app.py exposes interactive and operational surfaces including:
 - /control
 - /api/reviews/*
 - /api/tasks/active
 - /api/tools/*
 - /api/graph*
 - /feedback
 - /api/chat*
 - /voice
 - /calendar
 - /cor-unum*
- This indicates one consolidated runtime surface for review operations, task state, tool control, graph context, and user interaction channels.

2.2 Durable persistence and schema evolution

- noctem/db.py contains broad schema ownership for:
 - core domain objects (tasks/projects/goals)
 - agent execution/review queues
 - scheduler + delivery metadata
 - object/version/event tracking
 - plan step and conversation compaction data
- Includes migration and compatibility behavior plus reliability-oriented SQLite practices (including WAL usage patterns).

2.3 Queue-first agent execution and human review gating

- Primary queue/runtime path:
 - noctem/services/execution_queue.py
 - noctem/agent/execution_queue_runtime.py
- Review, orchestration, and interrupts:
 - noctem/agent/workflow.py

- noctem/agent/review_queue.py
- noctem/agent/plan_tracker.py
- noctem/agent/router.py
- noctem/agent/chat_orchestrator.py
- Overall mechanism: work is queued, orchestrated, and review-mediated rather than executed as uncontrolled direct action.

2.4 MCP tooling surface and deterministic operations

- MCP infrastructure:
 - noctem/mcp/server.py
 - noctem/mcp/contracts.py
 - noctem/mcp/resolver.py
 - noctem/mcp/tools.py
- Tool set covers:
 - CRUD for task/project/goal entities
 - deterministic resolution for object selection
 - bulk preview/commit flows
 - dependency/subtask operations
 - audit and undo behavior
 - import/export, health/version/capability endpoints

2.5 Memory, retrieval, and compacted context

- Context packing with explicit token budgeting:
 - noctem/agent/memory_pack.py
- Compaction and persistence of extracted conversational facts:
 - noctem/agent/compaction.py
- Retrieval stack:
 - noctem/wiki/retrieval.py
 - noctem/services/object_context_docs.py
 - noctem/services/object_graph.py
- Mechanism pattern: deterministic budgeted context + persisted summaries + retrieval from trusted object/wiki sources.

2.6 Background scheduling and delivery

- Scheduler gating logic:
 - noctem/scheduler/jobs.py

- Asynchronous outbound delivery/notification:
 - noctem/services/async_delivery.py
- Pattern: gated background execution with cooldown/conditions plus async publishing paths.

2.7 Additional subsystems

- Cor Unum ingestion pipeline:
 - noctem/ingestion/service.py
 - noctem/ingestion/engine.py
- Voice subsystem:
 - noctem/voice/processing.py
 - Current implementation emphasis is transcription pipeline behavior.

3. PROJECT STRUCTURE (PRACTICAL OVERVIEW)

- noctem/: core runtime/application modules (agent, web, mcp, db, scheduler, ingestion, voice, wiki, services)
- noctem/tests/: package-level tests targeting core internals
- tests/: broader suite with root-level test setup and compatibility coverage
- docs (parent): planning/history and project documentation artifacts

4. TESTING INFRASTRUCTURE AND OBSERVED BASELINES

4.1 Test framework and configuration

- Framework: pytest.
- Root configuration: tests/conftest.py
 - includes collection ignore behavior for selected suites
 - shared temporary database setup behavior
- Package configuration: noctem/tests/conftest.py
 - per-test temp DB fixtures and package test setup logic

4.2 Executions observed

- python -m pytest tests --collect-only -q
→ 311 collected
- python -m pytest noctem/tests --collect-only -q
→ 209 collected
- python -m pytest noctem/tests -q
→ 209 passed in 33.79s
- python -m pytest tests -q
→ exit code 0 (successful run), with captured output showing progress lines through and beyond

51%

→ final textual summary line was not present in captured output excerpt

5. DOCS VS CURRENT IMPLEMENTATION

- ../docs/OUTDATED/Plan 0.9.4.md is historical and should not be used as current behavioral truth.
- ../docs/Plan 0.9.4.1.md aligns with current direction at a high level, but implementation details should continue to be validated against source.
- Some comments/docstrings in source still reference older versions (for example 0.5/0.6/0.9.3-era labels), while active mechanics are newer; behavior in code takes precedence over version labels.

6. CURRENT-STATE CONCLUSION

The current version is a durable, queue-driven, review-gated agent platform with:

- broad MCP tool operations,
- persistent memory/retrieval and object-graph context mechanisms,
- gated scheduler and async delivery capabilities,
- ingestion and voice-transcription integration surfaces,
- and actively functioning pytest infrastructure with successful package and root-suite executions observed.